



Yuva Internship 2026

Week 1: Data Exploration and Problem Definition

Project Title

Exploratory Data Analysis of Online Retail Transactions

Intern

Gupteswara Sahu

Internship Role

Virtual Data Analysis Apprentice Intern

Tools Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Google Colab

In []: *# Import Libraries*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In []: *# Load Dataset*

```
df = pd.read_csv(
    "/content/OnlineRetail.csv",
    encoding="ISO-8859-1"
)
```

In []: *# Display first 5 rows*

```
df.head()
```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T- LIGHT HOLDER	6	01-12-2010 08:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	01-12-2010 08:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01-12-2010 08:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01-12-2010 08:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01-12-2010 08:26	3.39	17850.0	United Kingdom

In []:

```
df.shape
```

Out[]:

```
(541909, 8)
```

In []:

```
df.columns
```

Out[]:

```
Index(['InvoiceNo', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',  
      'UnitPrice', 'CustomerID', 'Country'],  
      dtype='object')
```

In []:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 541909 entries, 0 to 541908  
Data columns (total 8 columns):  
#   Column          Non-Null Count  Dtype  
---  -  
0   InvoiceNo       541909 non-null object  
1   StockCode      541909 non-null object  
2   Description     540455 non-null object  
3   Quantity       541909 non-null int64  
4   InvoiceDate     541909 non-null object  
5   UnitPrice      541909 non-null float64  
6   CustomerID     406829 non-null float64  
7   Country        541909 non-null object  
dtypes: float64(2), int64(1), object(5)  
memory usage: 33.1+ MB
```

```
In [ ]: df.describe()
```

```
Out[ ]:
```

	Quantity	UnitPrice	CustomerID
count	541909.000000	541909.000000	406829.000000
mean	9.552250	4.611114	15287.690570
std	218.081158	96.759853	1713.600303
min	-80995.000000	-11062.060000	12346.000000
25%	1.000000	1.250000	13953.000000
50%	3.000000	2.080000	15152.000000
75%	10.000000	4.130000	16791.000000
max	80995.000000	38970.000000	18287.000000

```
In [ ]: df.isnull().sum()
```

```
Out[ ]:
```

	0
InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135080
Country	0

dtype: int64

```
In [ ]: df.duplicated().sum()
```

```
Out[ ]: np.int64(5268)
```

2. Dataset Understanding

The Online Retail dataset contains transactional records from an online retail business. Each row represents a transaction-level record containing information about the invoice, product, quantity, price, customer, date, and country.

The purpose of this section is to understand the structure, variables, data types, and overall characteristics of the dataset before performing data cleaning and exploratory analysis.

```
In [ ]: # Display Column Names

for column in df.columns:
    print(column)
```

```
InvoiceNo
StockCode
Description
Quantity
InvoiceDate
UnitPrice
CustomerID
Country
```

```
In [ ]: # Check Data Types

df.dtypes
```

```
Out[ ]:      0
InvoiceNo  object
StockCode  object
Description object
Quantity   int64
InvoiceDate object
UnitPrice  float64
CustomerID float64
Country    object
```

dtype: object

```
In [ ]: # Count Unique Values in Each Column

df.nunique()
```

Out[]: 0

InvoiceNo	25900
StockCode	4070
Description	4223
Quantity	722
InvoiceDate	23260
UnitPrice	1630
CustomerID	4372
Country	38

dtype: int64

Dataset Variables

Variable	Description	Data Type
InvoiceNo	Unique invoice/transaction identifier	Categorical
StockCode	Unique product identifier	Categorical
Description	Description/name of the product	Text
Quantity	Number of units involved in the transaction	Numerical
InvoiceDate	Date and time of the transaction	Datetime
UnitPrice	Price per unit of the product	Numerical
CustomerID	Unique identifier of the customer	Categorical
Country	Country associated with the customer	Categorical

3. Data Quality Analysis

3.1 Missing Value Analysis

```
In [ ]: # Missing Value Count

missing_values = df.isnull().sum()

missing_values
```

```
Out[ ]:
```

	0
InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	135080
Country	0

dtype: int64

```
In [ ]: # Missing Value Percentage

missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_table = pd.DataFrame({
    "Missing Values": missing_values,
    "Missing Percentage": missing_percentage
})

missing_table.sort_values(
    by="Missing Percentage",
    ascending=False
)
```

```
Out[ ]:
```

	Missing Values	Missing Percentage
CustomerID	135080	24.926694
Description	1454	0.268311
StockCode	0	0.000000
InvoiceNo	0	0.000000
Quantity	0	0.000000
InvoiceDate	0	0.000000
UnitPrice	0	0.000000
Country	0	0.000000

```
In [ ]: # Number of records without Customer ID

df["CustomerID"].isnull().sum()
```

```
Out[ ]: np.int64(135080)
```

```
In [ ]: # Percentage of records without Customer ID

df["CustomerID"].isnull().mean() * 100
```

Out[]: np.float64(24.926694334288598)

```
In [ ]: # View records with missing Customer ID

df[df["CustomerID"].isnull()].head()
```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Cour
622	536414	22139	NaN	56	01-12-2010 11:52	0.00	NaN	Un King
1443	536544	21773	DECORATIVE ROSE BATHROOM BOTTLE	1	01-12-2010 14:32	2.51	NaN	Un King
1444	536544	21774	DECORATIVE CATS BATHROOM BOTTLE	2	01-12-2010 14:32	2.51	NaN	Un King
1445	536544	21786	POLKADOT RAIN HAT	4	01-12-2010 14:32	0.85	NaN	Un King
1446	536544	21787	RAIN PONCHO RETROSPOT	2	01-12-2010 14:32	1.66	NaN	Un King

```
In [ ]: # View records with missing Customer ID

df[df["CustomerID"].isnull()].head()
```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Cour
622	536414	22139	NaN	56	01-12-2010 11:52	0.00	NaN	Un King
1443	536544	21773	DECORATIVE ROSE BATHROOM BOTTLE	1	01-12-2010 14:32	2.51	NaN	Un King
1444	536544	21774	DECORATIVE CATS BATHROOM BOTTLE	2	01-12-2010 14:32	2.51	NaN	Un King
1445	536544	21786	POLKADOT RAIN HAT	4	01-12-2010 14:32	0.85	NaN	Un King
1446	536544	21787	RAIN PONCHO RETROSPOT	2	01-12-2010 14:32	1.66	NaN	Un King

In []: *# Display duplicate records*

```
df[df.duplicated()].head(10)
```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
--	-----------	-----------	-------------	----------	-------------	-----------	------------	---------

517	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	01-12-2010 11:45	1.25	17908.0	United Kingdom
527	536409	22866	HAND WARMER SCOTTY DOG DESIGN	1	01-12-2010 11:45	2.10	17908.0	United Kingdom
537	536409	22900	SET 2 TEA TOWELS I LOVE LONDON	1	01-12-2010 11:45	2.95	17908.0	United Kingdom
539	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	01-12-2010 11:45	4.95	17908.0	United Kingdom
555	536412	22327	ROUND SNACK BOXES SET OF 4 SKULLS	1	01-12-2010 11:49	2.95	17920.0	United Kingdom
587	536412	22273	FELTCRAFT DOLL MOLLY	1	01-12-2010 11:49	2.95	17920.0	United Kingdom
589	536412	22749	FELTCRAFT PRINCESS CHARLOTTE DOLL	1	01-12-2010 11:49	3.75	17920.0	United Kingdom
594	536412	22141	CHRISTMAS CRAFT TREE TOP ANGEL	1	01-12-2010 11:49	2.10	17920.0	United Kingdom
598	536412	21448	12 DAISY PEGS IN WOOD BOX	1	01-12-2010 11:49	1.65	17920.0	United Kingdom
600	536412	22569	FELTCRAFT CUSHION BUTTERFLY	2	01-12-2010 11:49	3.75	17920.0	United Kingdom

In []: df["InvoiceDate"].dtype

```
df["InvoiceDate"].dtype
```

Out[]: dtype('O')

In []: *# Quantity Summary*

```
df["Quantity"].describe()
```

Out[]: **Quantity**

count	541909.000000
mean	9.552250
std	218.081158
min	-80995.000000
25%	1.000000
50%	3.000000
75%	10.000000
max	80995.000000

dtype: float64

```
In [ ]: # Count records with zero or negative quantity  
(df["Quantity"] <= 0).sum()
```

Out[]: np.int64(10624)

```
In [ ]: # Count records with zero or negative quantity  
(df["Quantity"] <= 0).sum()
```

Out[]: np.int64(10624)

```
In [ ]: # View unusual quantity records  
df[df["Quantity"] <= 0].head(10)
```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
141	C536379	D	Discount	-1	01-12-2010 09:41	27.50	14527.0	United Kingdom
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	01-12-2010 09:49	4.65	15311.0	United Kingdom
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	01-12-2010 10:24	1.65	17548.0	United Kingdom
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	01-12-2010 10:24	0.29	17548.0	United Kingdom
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	01-12-2010 10:24	0.29	17548.0	United Kingdom
238	C536391	21980	PACK OF 12 RED RETROSPOT TISSUES	-24	01-12-2010 10:24	0.29	17548.0	United Kingdom
239	C536391	21484	CHICK GREY HOT WATER BOTTLE	-12	01-12-2010 10:24	3.45	17548.0	United Kingdom
240	C536391	22557	PLASTERS IN TIN VINTAGE PAISLEY	-12	01-12-2010 10:24	1.65	17548.0	United Kingdom
241	C536391	22553	PLASTERS IN TIN SKULLS	-24	01-12-2010 10:24	1.65	17548.0	United Kingdom
939	C536506	22960	JAM MAKING SET WITH JARS	-6	01-12-2010 12:38	4.25	17897.0	United Kingdom

In []:

```
df["Quantity"].min(), df["Quantity"].max()
```

Out[]:

```
(-80995, 80995)
```

In []:

```
# Unit Price Summary  
df["UnitPrice"].describe()
```

Out[]: **UnitPrice**

count	541909.000000
mean	4.611114
std	96.759853
min	-11062.060000
25%	1.250000
50%	2.080000
75%	4.130000
max	38970.000000

dtype: float64

In []: *# Count zero or negative prices*

```
(df["UnitPrice"] <= 0).sum()
```

Out[]: np.int64(2517)

In []: *# View records with zero or negative prices*

```
df[df["UnitPrice"] <= 0].head(10)
```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Cour
--	-----------	-----------	-------------	----------	-------------	-----------	------------	------

622	536414	22139	NaN	56	01-12-2010 11:52	0.0	NaN	Un Kingo
1970	536545	21134	NaN	1	01-12-2010 14:32	0.0	NaN	Un Kingo
1971	536546	22145	NaN	1	01-12-2010 14:33	0.0	NaN	Un Kingo
1972	536547	37509	NaN	1	01-12-2010 14:33	0.0	NaN	Un Kingo
1987	536549	85226A	NaN	1	01-12-2010 14:34	0.0	NaN	Un Kingo
1988	536550	85044	NaN	1	01-12-2010 14:34	0.0	NaN	Un Kingo
2024	536552	20950	NaN	1	01-12-2010 14:34	0.0	NaN	Un Kingo
2025	536553	37461	NaN	3	01-12-2010 14:35	0.0	NaN	Un Kingo
2026	536554	84670	NaN	23	01-12-2010 14:35	0.0	NaN	Un Kingo
2406	536589	21777	NaN	-10	01-12-2010 16:50	0.0	NaN	Un Kingo

In []: `df["UnitPrice"].min(), df["UnitPrice"].max()`

Out[]: `(-11062.06, 38970.0)`

In []: `# Identify cancelled transactions`

`cancelled = df["InvoiceNo"].astype(str).str.startswith("C")`

`cancelled.sum()`

Out[]: `np.int64(9288)`

In []: `# Percentage of cancelled transactions`

`cancelled.mean() * 100`

Out[]: `np.float64(1.7139409015166756)`

In []: `# View cancelled transactions`

`df[cancelled].head(10)`

Out[]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Count
--	-----------	-----------	-------------	----------	-------------	-----------	------------	-------

141	C536379	D	Discount	-1	01-12-2010 09:41	27.50	14527.0	Unit Kingdo
154	C536383	35004C	SET OF 3 COLOURED FLYING DUCKS	-1	01-12-2010 09:49	4.65	15311.0	Unit Kingdo
235	C536391	22556	PLASTERS IN TIN CIRCUS PARADE	-12	01-12-2010 10:24	1.65	17548.0	Unit Kingdo
236	C536391	21984	PACK OF 12 PINK PAISLEY TISSUES	-24	01-12-2010 10:24	0.29	17548.0	Unit Kingdo
237	C536391	21983	PACK OF 12 BLUE PAISLEY TISSUES	-24	01-12-2010 10:24	0.29	17548.0	Unit Kingdo
238	C536391	21980	PACK OF 12 RED RETROSPOT TISSUES	-24	01-12-2010 10:24	0.29	17548.0	Unit Kingdo
239	C536391	21484	CHICK GREY HOT WATER BOTTLE	-12	01-12-2010 10:24	3.45	17548.0	Unit Kingdo
240	C536391	22557	PLASTERS IN TIN VINTAGE PAISLEY	-12	01-12-2010 10:24	1.65	17548.0	Unit Kingdo
241	C536391	22553	PLASTERS IN TIN SKULLS	-24	01-12-2010 10:24	1.65	17548.0	Unit Kingdo
939	C536506	22960	JAM MAKING SET WITH JARS	-6	01-12-2010 12:38	4.25	17897.0	Unit Kingdo

In []: *# Create a copy for analysis*

```
df_analysis = df.copy()
```

In []: *# Separate cancelled transactions*

```
df_cancelled = df_analysis[
    df_analysis["InvoiceNo"].astype(str).str.startswith("C")
].copy()

df_sales = df_analysis[
    ~df_analysis["InvoiceNo"].astype(str).str.startswith("C")
].copy()
```

In []: df_sales.shape

Out[]: (532621, 8)

```
In [ ]: df_cancelled.shape
```

Out[]: (9288, 8)

```
In [ ]: df_cancelled.shape
```

Out[]: (9288, 8)

```
In [ ]: # Calculate Revenue

df_sales["Revenue"] = df_sales["Quantity"] * df_sales["UnitPrice"]
```

```
In [ ]: df_sales["Revenue"].describe()
```

Out[]:

	Revenue
count	532621.000000
mean	19.985244
std	270.574241
min	-11062.060000
25%	3.750000
50%	9.900000
75%	17.700000
max	168469.600000

dtype: float64

```
In [ ]: # Total Revenue

df_sales["Revenue"].sum()
```

Out[]: np.float64(10644560.424000002)

```
In [ ]: # Check missing values after creating sales dataset

df_sales.isnull().sum()
```

```
Out[ ]:
```

	0
InvoiceNo	0
StockCode	0
Description	1454
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	134697
Country	0
Revenue	0

dtype: int64

```
In [ ]: # Check duplicate records
df_sales.duplicated().sum()
```

```
Out[ ]: np.int64(5231)
```

```
In [ ]: # Check negative revenue
(df_sales["Revenue"] < 0).sum()
```

```
Out[ ]: np.int64(2)
```

4. Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed to identify patterns, distributions, relationships, trends, and unusual observations within the dataset.

```
In [ ]: # Overall Sales Metrics

total_revenue = df_sales["Revenue"].sum()
total_transactions = df_sales["InvoiceNo"].nunique()
total_products = df_sales["StockCode"].nunique()
total_customers = df_sales["CustomerID"].nunique()

print("Total Revenue:", total_revenue)
print("Total Transactions:", total_transactions)
print("Total Products:", total_products)
print("Total Customers:", total_customers)
```

Total Revenue: 10644560.424000002
Total Transactions: 22064
Total Products: 4059
Total Customers: 4339

```
In [ ]: # Top 10 Products by Quantity Sold

top_products_quantity = (
    df_sales.groupby("Description")["Quantity"]
        .sum()
        .sort_values(ascending=False)
        .head(10)
)

top_products_quantity
```

```
Out[ ]:
```

	Quantity
Description	
PAPER CRAFT , LITTLE BIRDIE	80995
MEDIUM CERAMIC TOP STORAGE JAR	78033
WORLD WAR 2 GLIDERS ASSTD DESIGNS	55047
JUMBO BAG RED RETROSPOT	48478
WHITE HANGING HEART T-LIGHT HOLDER	37895
POPCORN HOLDER	36761
ASSORTED COLOUR BIRD ORNAMENT	36461
PACK OF 72 RETROSPOT CAKE CASES	36419
RABBIT NIGHT LIGHT	30788
MINI PAINT SET VINTAGE	26633

dtype: int64

```
In [ ]: # Top 10 Products by Revenue

top_products_revenue = df_sales.groupby("Description")["Revenue"].sum().s

top_products_revenue
```

Out[]:

	Revenue
Description	
DOTCOM POSTAGE	206248.77
REGENCY CAKESTAND 3 TIER	174484.74
PAPER CRAFT , LITTLE BIRDIE	168469.60
WHITE HANGING HEART T-LIGHT HOLDER	106292.77
PARTY BUNTING	99504.33
JUMBO BAG RED RETROSPOT	94340.05
MEDIUM CERAMIC TOP STORAGE JAR	81700.92
Manual	78112.82
POSTAGE	78101.88
RABBIT NIGHT LIGHT	66964.99

dtype: float64

In []: *# Revenue by Country*

```
country_revenue = (  
    df_sales.groupby("Country")["Revenue"]  
        .sum()  
        .sort_values(ascending=False)  
)  
  
country_revenue.head(10)
```

Out[]:

	Revenue
Country	
United Kingdom	9003097.964
Netherlands	285446.340
EIRE	283453.960
Germany	228867.140
France	209715.110
Australia	138521.310
Spain	61577.110
Switzerland	57089.900
Belgium	41196.340
Sweden	38378.330

dtype: float64

```
In [ ]: # Transactions by Country

country_transactions = (
    df_sales.groupby("Country")["InvoiceNo"]
    .nunique()
    .sort_values(ascending=False)
)

country_transactions.head(10)
```

Out[]:

InvoiceNo	
Country	
United Kingdom	20122
Germany	457
France	392
EIRE	288
Belgium	98
Netherlands	95
Spain	90
Portugal	58
Australia	57
Switzerland	54

dtype: int64

```
In [ ]: # Convert InvoiceDate to datetime and extract components
```

```
df_sales["InvoiceDate"] = pd.to_datetime(df_sales["InvoiceDate"], format=
df_sales["Year"] = df_sales["InvoiceDate"].dt.year
df_sales["Month"] = df_sales["InvoiceDate"].dt.month
df_sales["MonthName"] = df_sales["InvoiceDate"].dt.month_name()
```

```
In [ ]: # Extract Date Components
```

```
df_sales["Year"] = df_sales["InvoiceDate"].dt.year
df_sales["Month"] = df_sales["InvoiceDate"].dt.month
df_sales["MonthName"] = df_sales["InvoiceDate"].dt.month_name()
```

```
In [ ]: # Monthly Revenue
```

```
monthly_revenue = (
    df_sales
    .groupby(df_sales["InvoiceDate"].dt.to_period("M"))["Revenue"]
    .sum()
)

monthly_revenue
```

Out[]:

Revenue	
InvoiceDate	
2010-12	823746.140
2011-01	691364.560
2011-02	523631.890
2011-03	717639.360
2011-04	537808.621
2011-05	770536.020
2011-06	761739.900
2011-07	719221.191
2011-08	737014.260
2011-09	1058590.172
2011-10	1154979.300
2011-11	1509496.330
2011-12	638792.680

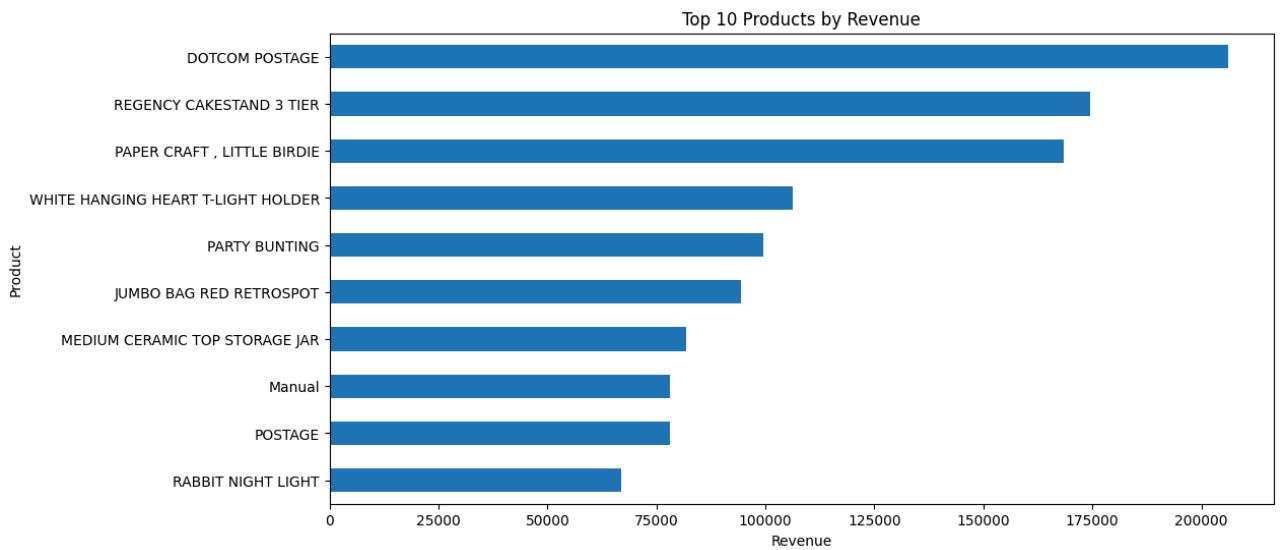
dtype: float64

```
In [ ]: plt.figure(figsize=(12, 6))

top_products_revenue.sort_values().plot(kind="barh")

plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")

plt.show()
```



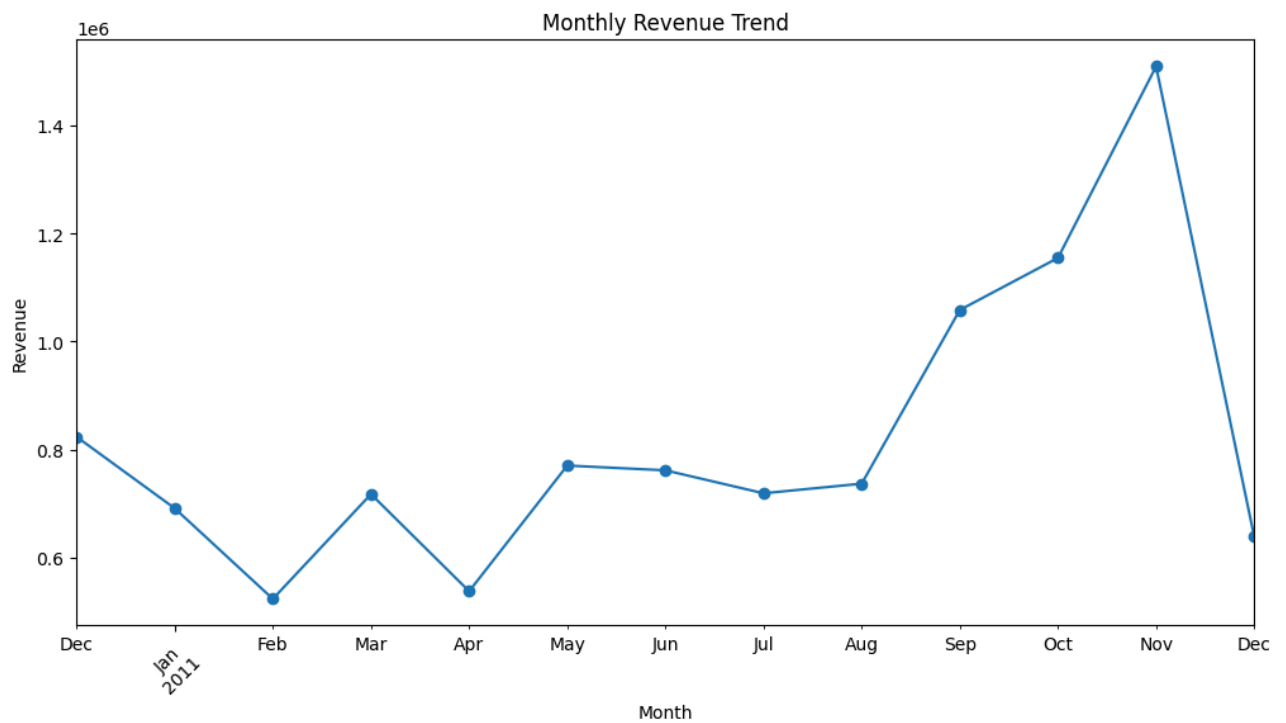
```
In [ ]: plt.figure(figsize=(12, 6))

monthly_revenue.plot(kind="line", marker="o")

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")

plt.xticks(rotation=45)

plt.show()
```



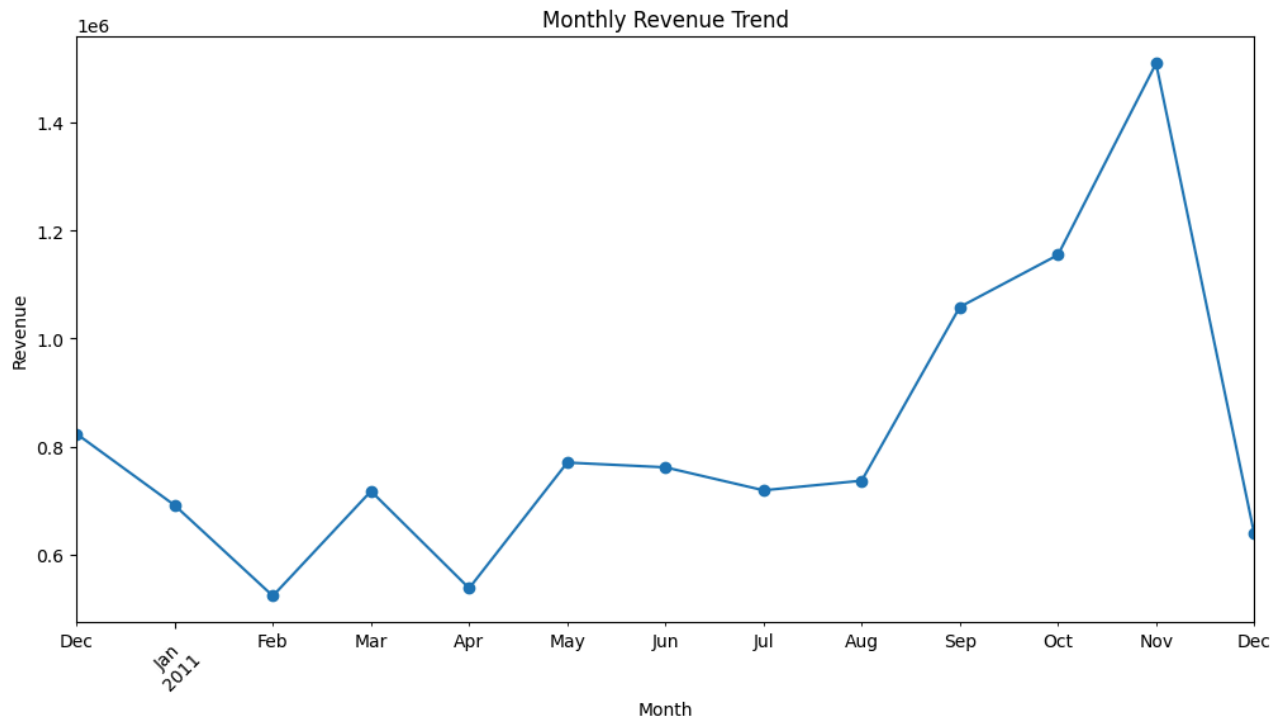
```
In [ ]: plt.figure(figsize=(12, 6))

monthly_revenue.plot(kind="line", marker="o")

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")

plt.xticks(rotation=45)

plt.show()
```

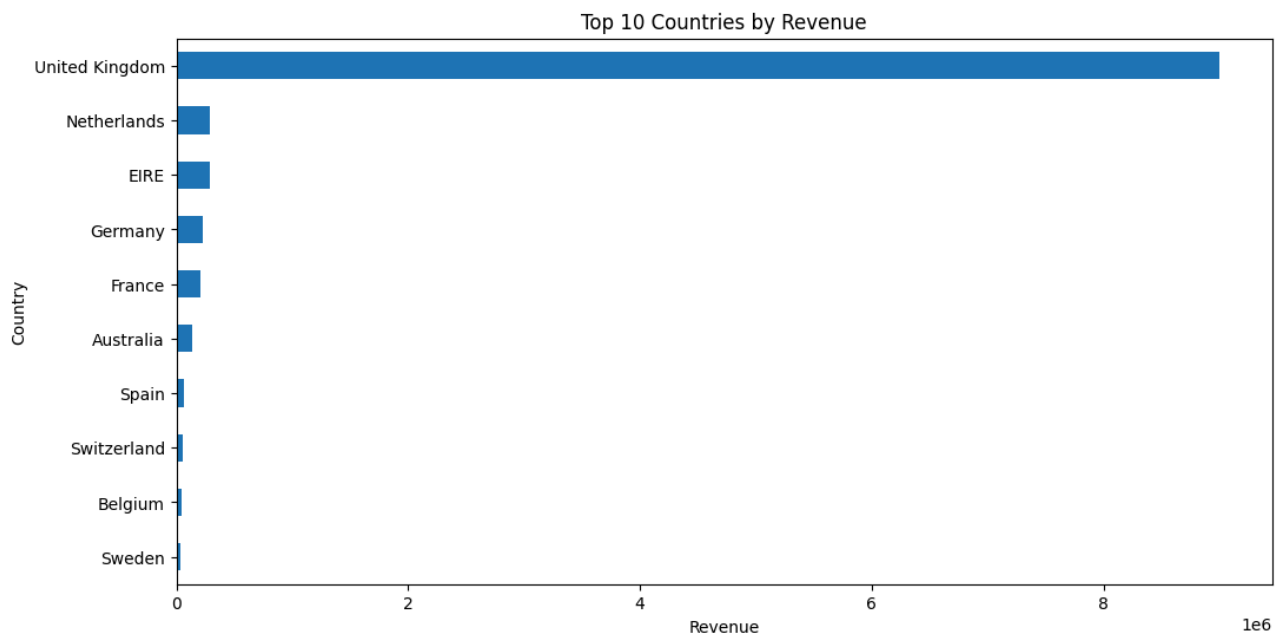


```
In [ ]: plt.figure(figsize=(12, 6))

country_revenue.head(10).sort_values().plot(kind="barh")

plt.title("Top 10 Countries by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Country")

plt.show()
```

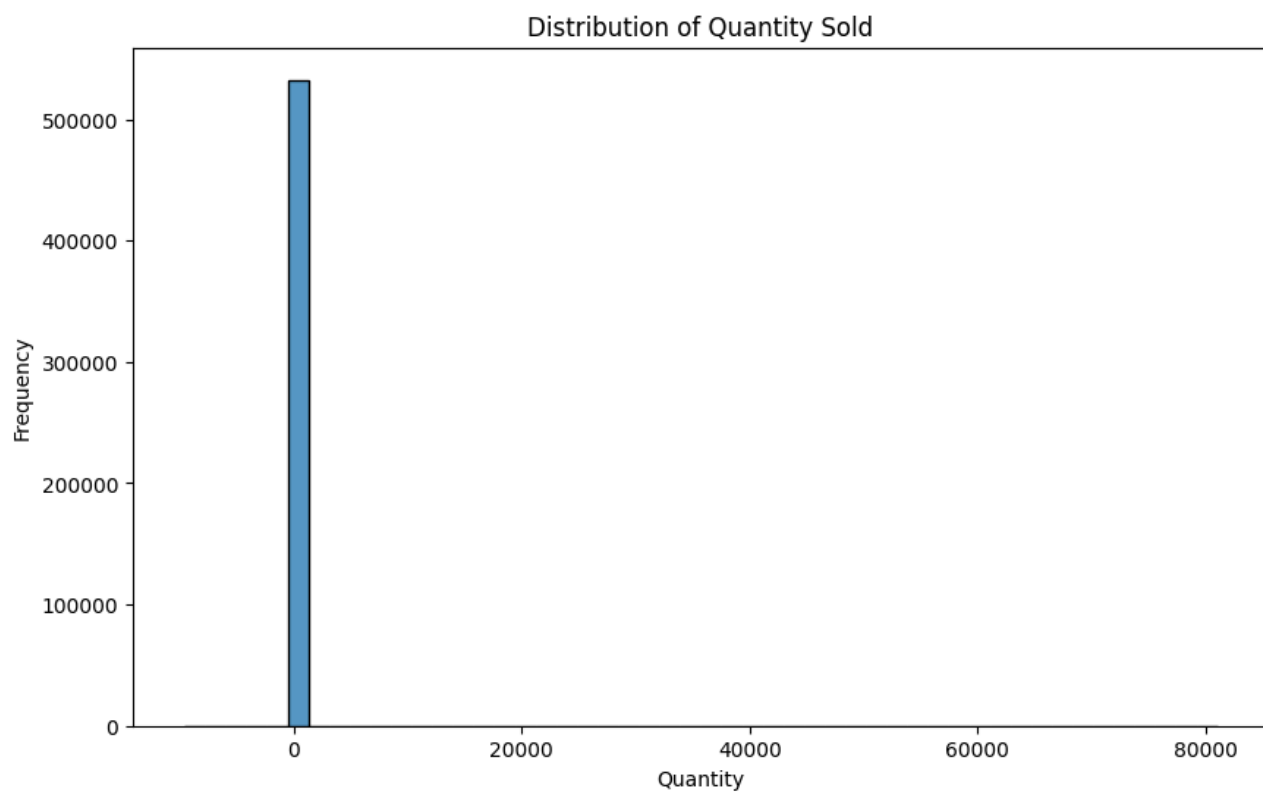


```
In [ ]: plt.figure(figsize=(10, 6))

sns.histplot(
    df_sales["Quantity"],
    bins=50
)

plt.title("Distribution of Quantity Sold")
plt.xlabel("Quantity")
plt.ylabel("Frequency")

plt.show()
```

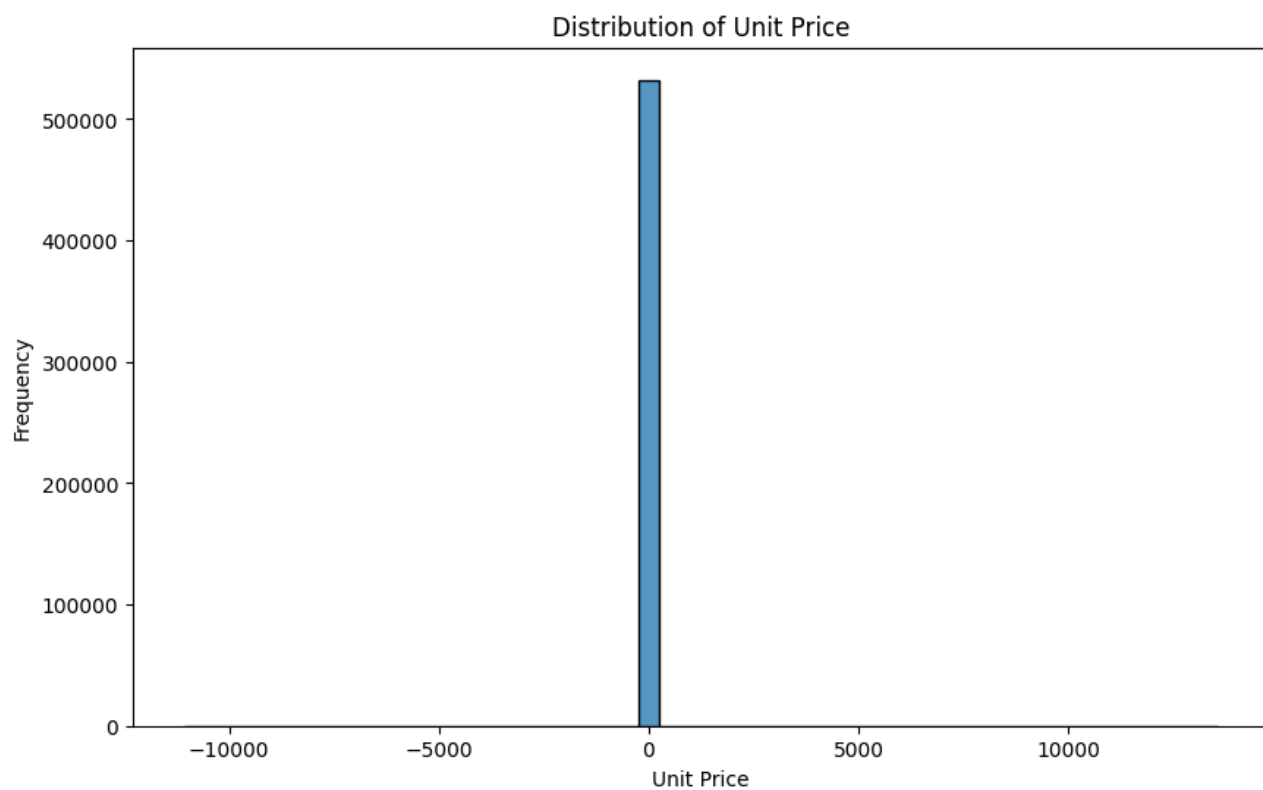


```
In [ ]: plt.figure(figsize=(10, 6))

sns.histplot(
    df_sales["UnitPrice"],
    bins=50
)

plt.title("Distribution of Unit Price")
plt.xlabel("Unit Price")
plt.ylabel("Frequency")

plt.show()
```

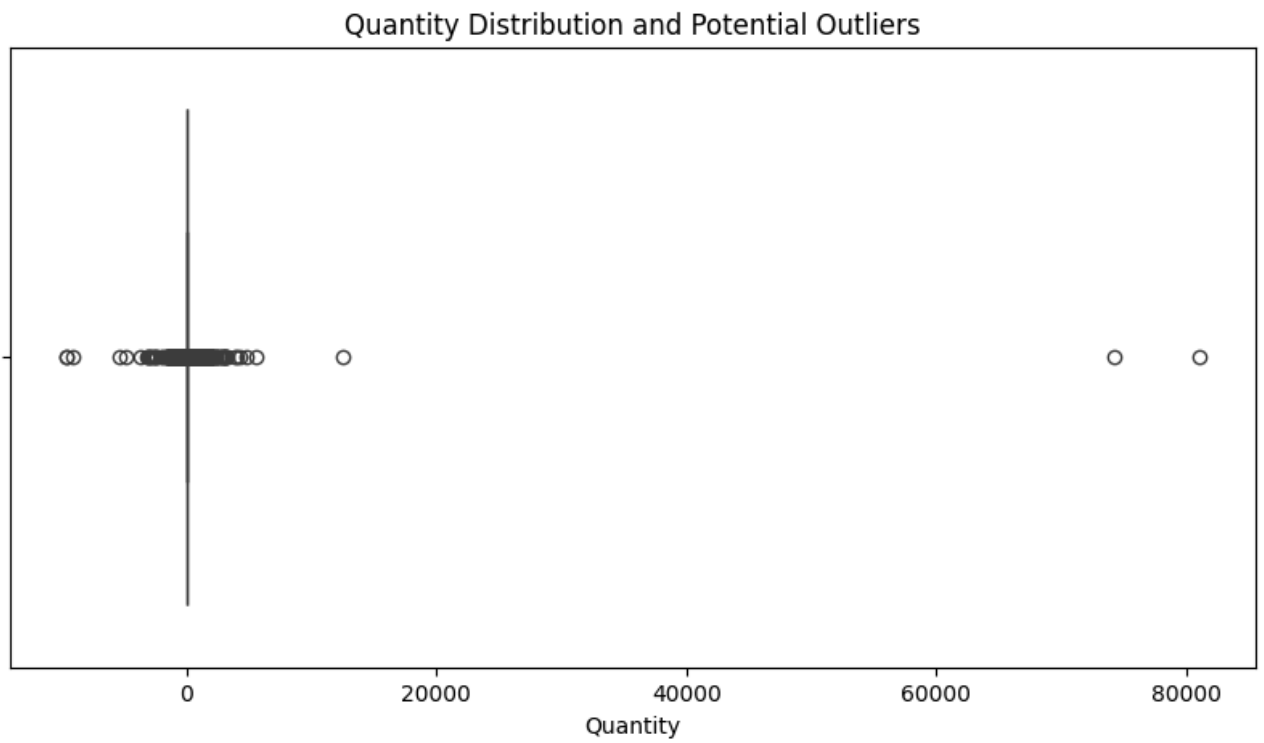


```
In [ ]: plt.figure(figsize=(10, 5))

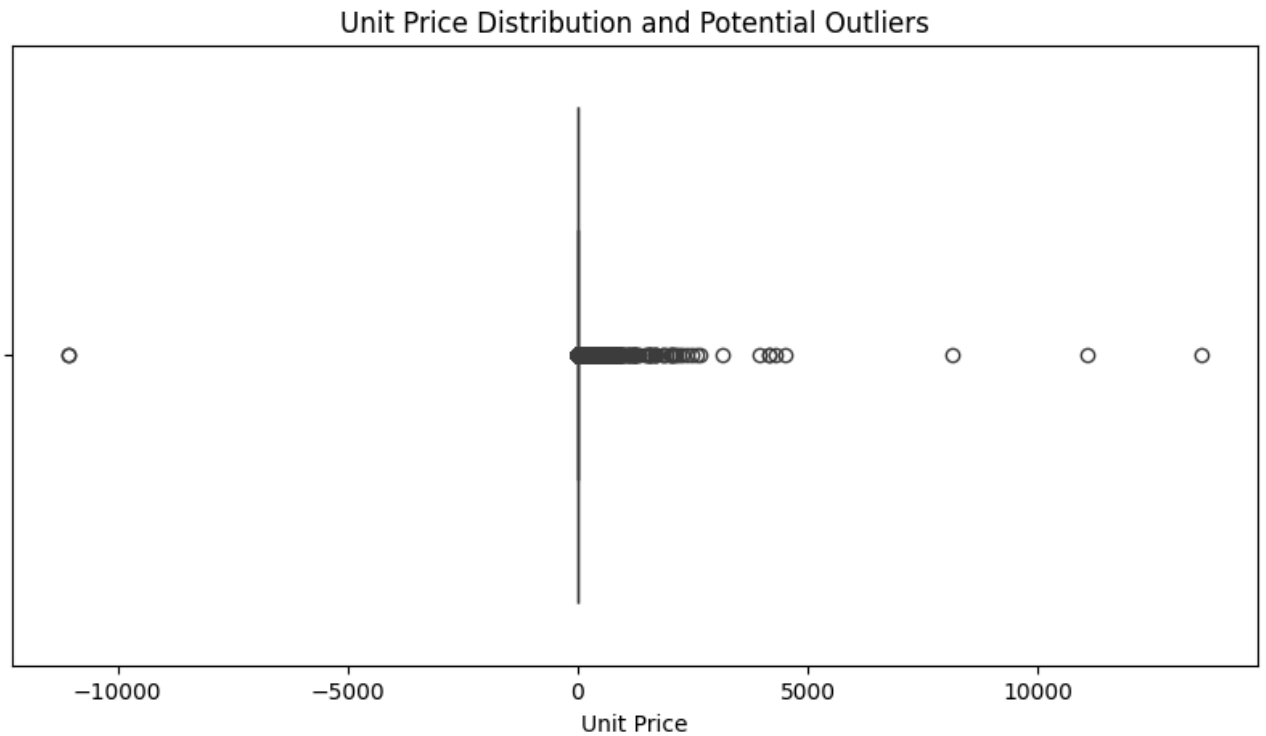
sns.boxplot(x=df_sales["Quantity"])

plt.title("Quantity Distribution and Potential Outliers")
plt.xlabel("Quantity")

plt.show()
```



```
In [ ]: plt.figure(figsize=(10, 5))  
sns.boxplot(x=df_sales["UnitPrice"])  
plt.title("Unit Price Distribution and Potential Outliers")  
plt.xlabel("Unit Price")  
plt.show()
```



5. Customer Analysis

Customer-level analysis is performed to understand purchasing behavior, customer contribution to revenue, and differences in transaction activity.

```
In [ ]: # Number of Unique Customers

unique_customers = df_sales["CustomerID"].nunique()

print("Number of Unique Customers:", unique_customers)
```

Number of Unique Customers: 4339

```
In [ ]: # Customer Revenue

customer_revenue = (
    df_sales
    .groupby("CustomerID")["Revenue"]
    .sum()
    .sort_values(ascending=False)
)

customer_revenue.head(10)
```

Out[]: **Revenue**

CustomerID	
14646.0	280206.02
18102.0	259657.30
17450.0	194550.79
16446.0	168472.50
14911.0	143825.06
12415.0	124914.53
14156.0	117379.63
17511.0	91062.38
16029.0	81024.84
12346.0	77183.60

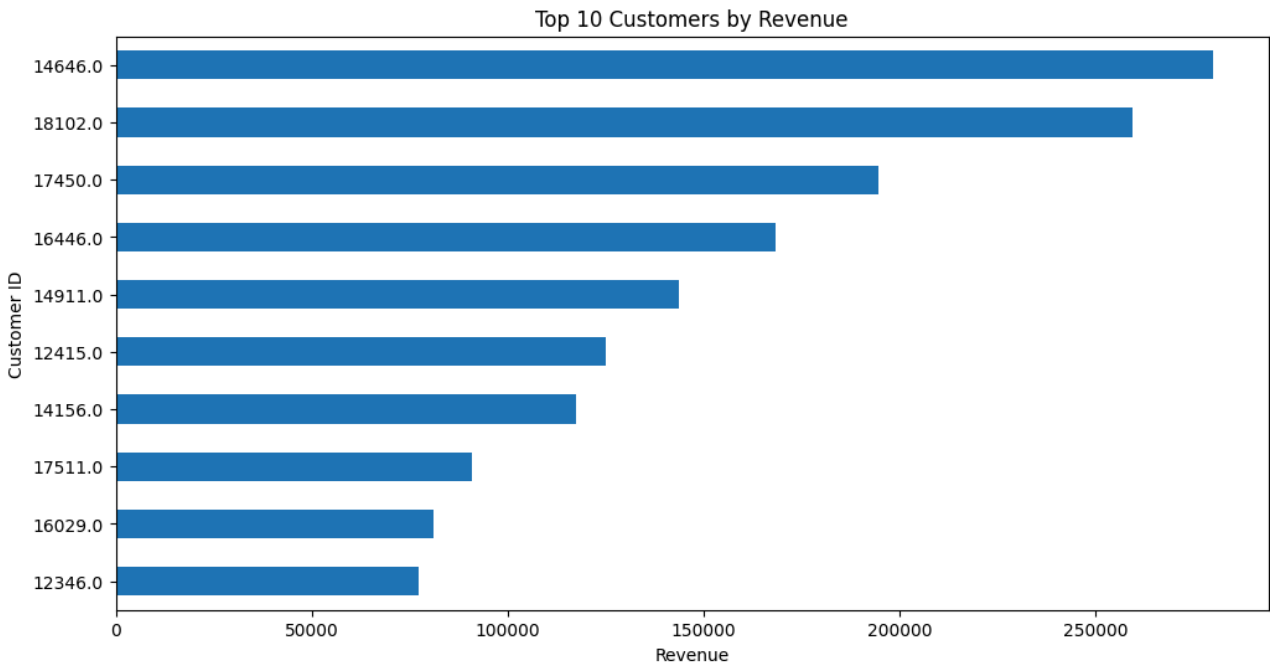
dtype: float64

```
In [ ]: plt.figure(figsize=(12, 6))

customer_revenue.head(10).sort_values().plot(kind="barh")

plt.title("Top 10 Customers by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Customer ID")

plt.show()
```



```
In [ ]: # Revenue per Invoice
```

```
invoice_revenue = (  
    df_sales  
    .groupby("InvoiceNo")["Revenue"]  
    .sum()  
)
```

```
In [ ]: # Average Order Value
```

```
average_order_value = invoice_revenue.mean()  
  
print("Average Order Value:", average_order_value)
```

Average Order Value: 482.44019325598265

```
In [ ]: print("Minimum Order Value:", invoice_revenue.min())  
        print("Maximum Order Value:", invoice_revenue.max())
```

Minimum Order Value: -11062.06

Maximum Order Value: 168469.6

```
In [ ]: # Number of Orders per Customer
```

```
customer_orders = (  
    df_sales  
    .groupby("CustomerID")["InvoiceNo"]  
    .nunique()  
    .sort_values(ascending=False)  
)  
  
customer_orders.head(10)
```

Out[]: **InvoiceNo**

CustomerID	
12748.0	210
14911.0	201
17841.0	124
13089.0	97
14606.0	93
15311.0	91
12971.0	86
14646.0	74
16029.0	63
13408.0	62

dtype: int64

```
In [ ]: print(  
        "Average Orders per Customer:",  
        customer_orders.mean()  
    )
```

Average Orders per Customer: 4.271952062687255

```
In [ ]: # Identify Repeat Customers  
  
repeat_customers = customer_orders[customer_orders > 1]  
  
print("Repeat Customers:", len(repeat_customers))
```

Repeat Customers: 2845

```
In [ ]: repeat_customer_percentage = (  
        len(repeat_customers) /  
        len(customer_orders)  
    ) * 100  
  
print(  
    "Repeat Customer Percentage:",  
    repeat_customer_percentage  
)
```

Repeat Customer Percentage: 65.56810324959667

```
In [ ]: total_customer_revenue = customer_revenue.sum()  
  
top_10_customer_revenue = customer_revenue.head(10).sum()  
  
top_10_percentage = (
```

```

    top_10_customer_revenue /
    total_customer_revenue
) * 100

print(
    "Percentage of Revenue Generated by Top 10 Customers:",
    top_10_percentage
)

```

Percentage of Revenue Generated by Top 10 Customers: 17.261881248972173

In []: *# Product Performance*

```

product_analysis = (
    df_sales
    .groupby("Description")
    .agg(
        Total_Quantity=("Quantity", "sum"),
        Total_Revenue=("Revenue", "sum"),
        Average_Price=("UnitPrice", "mean"),
        Number_of_Transactions=("InvoiceNo", "nunique")
    )
    .sort_values("Total_Revenue", ascending=False)
)

product_analysis.head(10)

```

Out[]:

	Total_Quantity	Total_Revenue	Average_Price	Number_of_Transactions
Description				
DOTCOM POSTAGE	708	206248.77	291.311822	708
REGENCY CAKESTAND 3 TIER	13890	174484.74	13.963081	1989
PAPER CRAFT , LITTLE BIRDIE	80995	168469.60	2.080000	1
WHITE HANGING HEART T-LIGHT HOLDER	37895	106292.77	3.211418	2260
PARTY BUNTING	18299	99504.33	5.790879	1686
JUMBO BAG RED RETROSPOT	48478	94340.05	2.481934	2092
MEDIUM CERAMIC TOP STORAGE JAR	78033	81700.92	1.468480	247
Manual	7231	78112.82	225.941402	295
POSTAGE	3150	78101.88	31.076581	1126
RABBIT NIGHT LIGHT	30788	66964.99	2.380502	994

```
In [ ]: product_analysis.sort_values(  
        "Total_Quantity",  
        ascending=False  
    ).head(10)
```

Out[]:

	Total_Quantity	Total_Revenue	Average_Price	Number_of_Transactions
Description				
PAPER CRAFT , LITTLE BIRDIE	80995	168469.60	2.080000	1
MEDIUM CERAMIC TOP STORAGE JAR	78033	81700.92	1.468480	247
WORLD WAR 2 GLIDERS ASSTD DESIGNS	55047	13841.85	0.320670	535
JUMBO BAG RED RETROSPOT	48478	94340.05	2.481934	2092
WHITE HANGING HEART T-LIGHT HOLDER	37895	106292.77	3.211418	2260
POPCORN HOLDER	36761	34298.87	1.020012	803
ASSORTED COLOUR BIRD ORNAMENT	36461	59094.93	1.722290	1455
PACK OF 72 RETROSPOT CAKE CASES	36419	21259.10	0.758431	1320
RABBIT NIGHT LIGHT	30788	66964.99	2.380502	994
MINI PAINT SET VINTAGE	26633	16937.82	0.785026	380

```
In [ ]: product_analysis.sort_values(  
        "Total_Revenue",  
        ascending=False  
    ).head(10)
```

Out[]:

	Total_Quantity	Total_Revenue	Average_Price	Number_of_Transactions
Description				
DOTCOM POSTAGE	708	206248.77	291.311822	708
REGENCY CAKESTAND 3 TIER	13890	174484.74	13.963081	1989
PAPER CRAFT , LITTLE BIRDIE	80995	168469.60	2.080000	1
WHITE HANGING HEART T- LIGHT HOLDER	37895	106292.77	3.211418	2260
PARTY BUNTING	18299	99504.33	5.790879	1686
JUMBO BAG RED RETROSPOT	48478	94340.05	2.481934	2092
MEDIUM CERAMIC TOP STORAGE JAR	78033	81700.92	1.468480	247
Manual	7231	78112.82	225.941402	295
POSTAGE	3150	78101.88	31.076581	1126
RABBIT NIGHT LIGHT	30788	66964.99	2.380502	994

In []:

```
# Country-Level Business Analysis

country_analysis = (
    df_sales
    .groupby("Country")
    .agg(
        Total_Revenue=("Revenue", "sum"),
        Total_Quantity=("Quantity", "sum"),
        Transactions=("InvoiceNo", "nunique"),
        Customers=("CustomerID", "nunique")
    )
    .sort_values("Total_Revenue", ascending=False)
)

country_analysis.head(10)
```

Out[]:

	Total_Revenue	Total_Quantity	Transactions	Customers
Country				
United Kingdom	9003097.964	4526862	20122	3921
Netherlands	285446.340	200937	95	9
EIRE	283453.960	147447	288	3
Germany	228867.140	119263	457	94
France	209715.110	112104	392	87
Australia	138521.310	84209	57	9
Spain	61577.110	27951	90	30
Switzerland	57089.900	30630	54	21
Belgium	41196.340	23237	98	25
Sweden	38378.330	36083	36	8

In []:

```
# Average Revenue per Transaction by Country

country_analysis["Average_Transaction_Value"] = (
    country_analysis["Total_Revenue"] /
    country_analysis["Transactions"]
)
```

In []:

```
country_analysis.sort_values(
    "Average_Transaction_Value",
    ascending=False
).head(10)
```

Out[]:

	Total_Revenue	Total_Quantity	Transactions	Customers	Average_Transaction_Va
Country					
Singapore	21279.29	5241	7	1	3039.898
Netherlands	285446.34	200937	95	9	3004.698
Australia	138521.31	84209	57	9	2430.198
Japan	37416.37	26016	19	8	1969.282
Lebanon	1693.88	386	1	1	1693.880
Hong Kong	15691.80	4773	11	0	1426.527
Brazil	1143.60	356	1	1	1143.600
Sweden	38378.33	36083	36	8	1066.064
Switzerland	57089.90	30630	54	21	1057.220
Denmark	18955.34	8235	18	9	1053.074

```
In [ ]: # Monthly Business Analysis

monthly_analysis = (
    df_sales
    .groupby(df_sales["InvoiceDate"].dt.to_period("M"))
    .agg(
        Revenue=("Revenue", "sum"),
        Transactions=("InvoiceNo", "nunique"),
        Quantity=("Quantity", "sum")
    )
)

monthly_analysis
```

```
Out[ ]:
```

	Revenue	Transactions	Quantity
InvoiceDate			
2010-12	823746.140	1699	358270
2011-01	691364.560	1216	388811
2011-02	523631.890	1174	280860
2011-03	717639.360	1665	357267
2011-04	537808.621	1504	302743
2011-05	770536.020	1848	386259
2011-06	761739.900	1683	348985
2011-07	719221.191	1657	397144
2011-08	737014.260	1459	418545
2011-09	1058590.172	1994	557246
2011-10	1154979.300	2275	595073
2011-11	1509496.330	3021	752907
2011-12	638792.680	869	309914

```
In [ ]: # Monthly Average Order Value

monthly_analysis["Average_Order_Value"] = (
    monthly_analysis["Revenue"] /
    monthly_analysis["Transactions"]
)

monthly_analysis
```

Out[]:

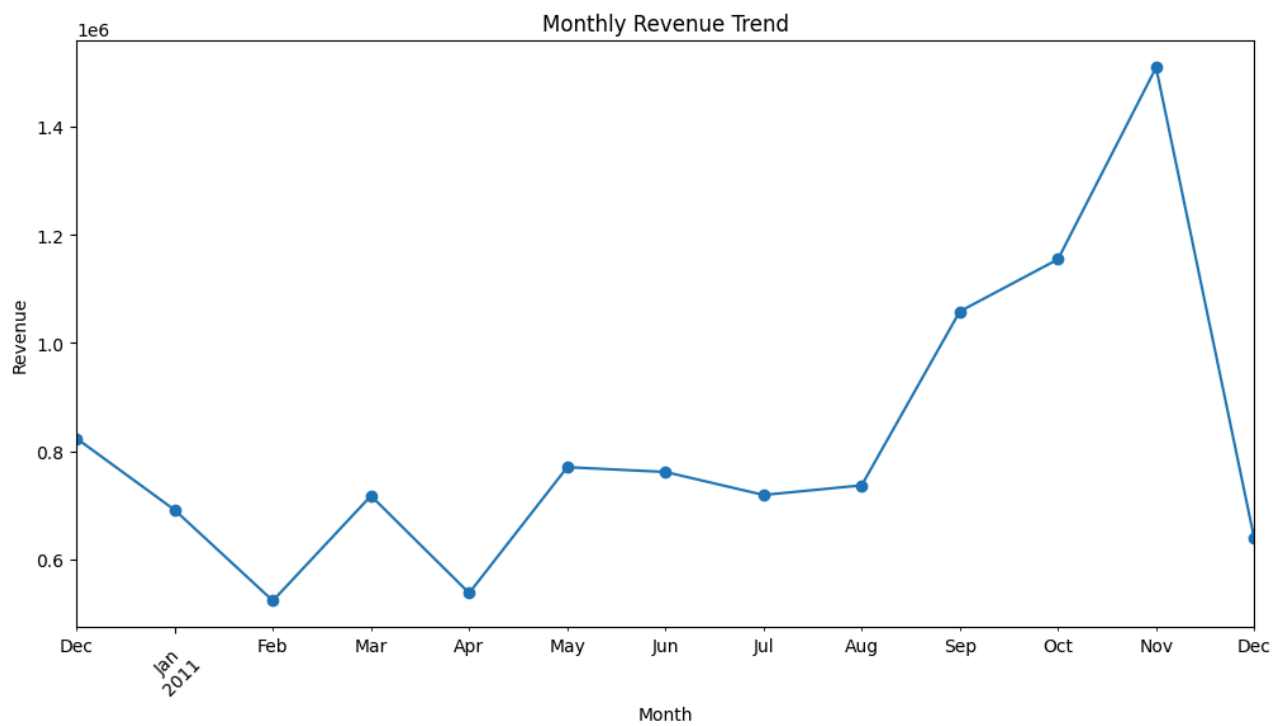
	Revenue	Transactions	Quantity	Average_Order_Value
InvoiceDate				
2010-12	823746.140	1699	358270	484.841754
2011-01	691364.560	1216	388811	568.556382
2011-02	523631.890	1174	280860	446.023756
2011-03	717639.360	1665	357267	431.014631
2011-04	537808.621	1504	302743	357.585519
2011-05	770536.020	1848	386259	416.956721
2011-06	761739.900	1683	348985	452.608378
2011-07	719221.191	1657	397144	434.050206
2011-08	737014.260	1459	418545	505.150281
2011-09	1058590.172	1994	557246	530.887749
2011-10	1154979.300	2275	595073	507.683209
2011-11	1509496.330	3021	752907	499.667769
2011-12	638792.680	869	309914	735.089390

```
In [ ]: plt.figure(figsize=(12, 6))

monthly_analysis["Revenue"].plot(
    kind="line",
    marker="o"
)

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.xticks(rotation=45)

plt.show()
```

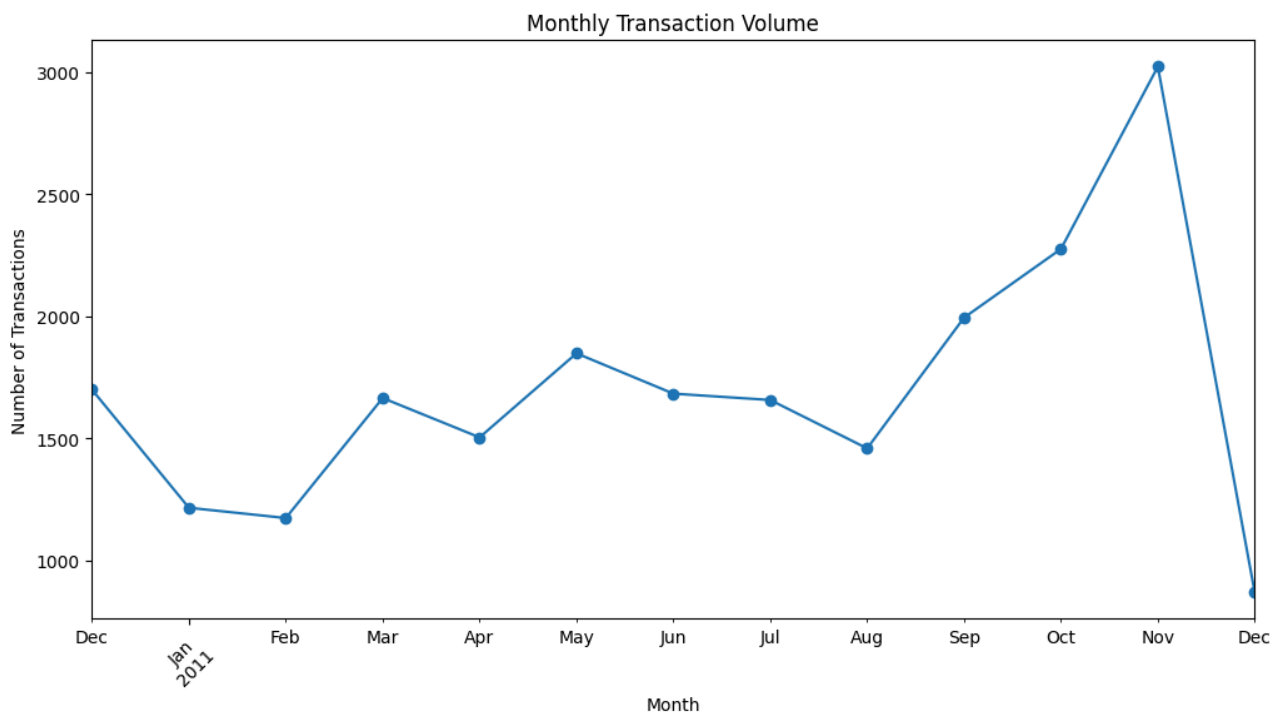


```
In [ ]: plt.figure(figsize=(12, 6))

monthly_analysis["Transactions"].plot(
    kind="line",
    marker="o"
)

plt.title("Monthly Transaction Volume")
plt.xlabel("Month")
plt.ylabel("Number of Transactions")
plt.xticks(rotation=45)

plt.show()
```

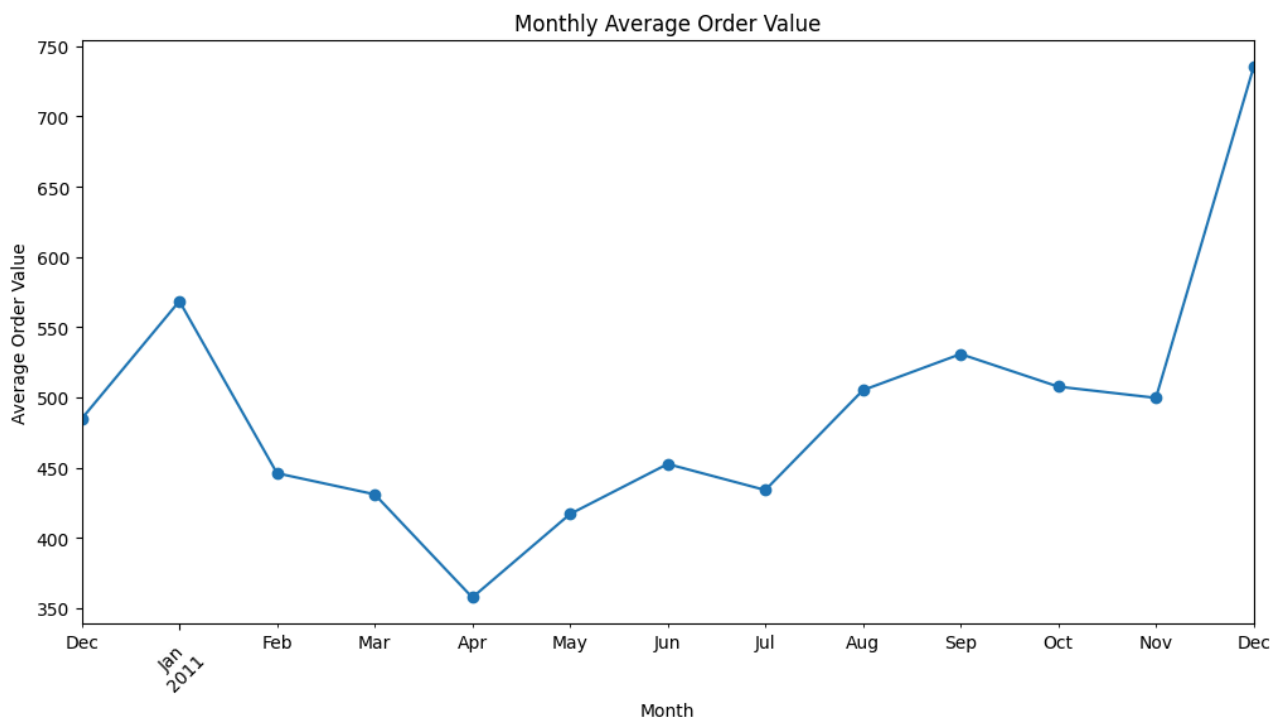


```
In [ ]: plt.figure(figsize=(12, 6))

monthly_analysis["Average_Order_Value"].plot(
    kind="line",
    marker="o"
)

plt.title("Monthly Average Order Value")
plt.xlabel("Month")
plt.ylabel("Average Order Value")
plt.xticks(rotation=45)

plt.show()
```



6. Cancellation Analysis

Cancelled transactions are analyzed separately to understand their frequency and potential impact on the retail dataset.

```
In [ ]: # Cancellation Summary

cancelled_count = len(df_cancelled)

total_records = len(df)

cancellation_percentage = (
    cancelled_count / total_records
) * 100

print("Cancelled Records:", cancelled_count)
print("Cancellation Percentage:", cancellation_percentage)
```

Cancelled Records: 9288
Cancellation Percentage: 1.7139409015166756

```
In [ ]: # Total Cancelled Quantity

cancelled_quantity = df_cancelled["Quantity"].abs().sum()

print(
    "Total Cancelled Quantity:",
    cancelled_quantity
)
```

Total Cancelled Quantity: 277574

```
In [ ]: # Most Cancelled Products

cancelled_products = (
    df_cancelled
    .groupby("Description")["Quantity"]
    .sum()
    .abs()
    .sort_values(ascending=False)
    .head(10)
)

cancelled_products
```

```
Out[ ]:
```

	Quantity
Description	
PAPER CRAFT , LITTLE BIRDIE	80995
MEDIUM CERAMIC TOP STORAGE JAR	74494
ROTATING SILVER ANGELS T-LIGHT HLDR	9376
Manual	4066
FAIRY CAKE FLANNEL ASSORTED COLOUR	3150
PINK BLUE FELT CRAFT TRINKET BOX	2617
WHITE HANGING HEART T-LIGHT HOLDER	2578
GIN + TONIC DIET METAL SIGN	2030
HERB MARKER BASIL	1527
FELTCRAFT DOLL MOLLY	1447

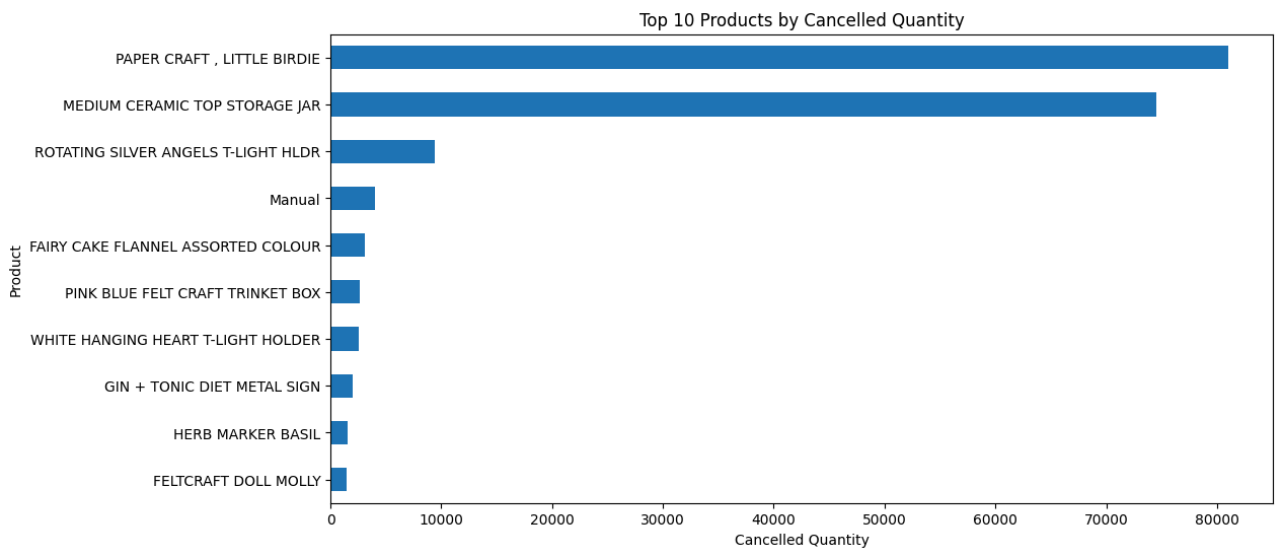
dtype: int64

```
In [ ]: plt.figure(figsize=(12, 6))

cancelled_products.sort_values().plot(kind="barh")

plt.title("Top 10 Products by Cancelled Quantity")
plt.xlabel("Cancelled Quantity")
plt.ylabel("Product")

plt.show()
```



7. Relationship Analysis

In []: *# Select Numerical Variables*

```
numeric_columns = [
    "Quantity",
    "UnitPrice",
    "Revenue"
]

correlation_matrix = df_sales[numeric_columns].corr()

correlation_matrix
```

Out[]:

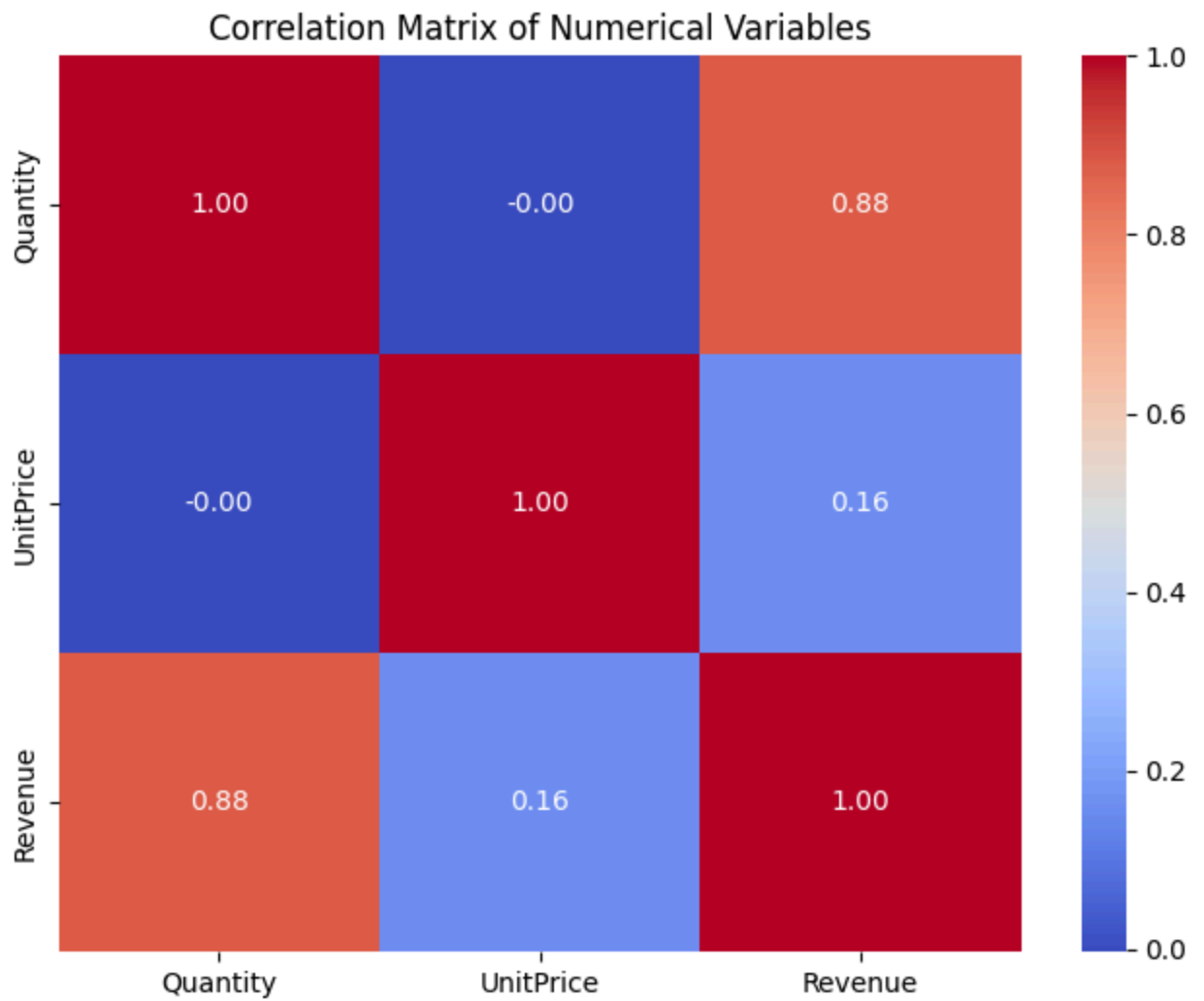
	Quantity	UnitPrice	Revenue
Quantity	1.000000	-0.002913	0.879465
UnitPrice	-0.002913	1.000000	0.158315
Revenue	0.879465	0.158315	1.000000

```
In [ ]: plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix of Numerical Variables")

plt.show()
```



```
In [ ]: df_sales["Revenue"].describe()
```

```
Out[ ]:
```

	Revenue
count	532621.000000
mean	19.985244
std	270.574241
min	-11062.060000
25%	3.750000
50%	9.900000
75%	17.700000
max	168469.600000

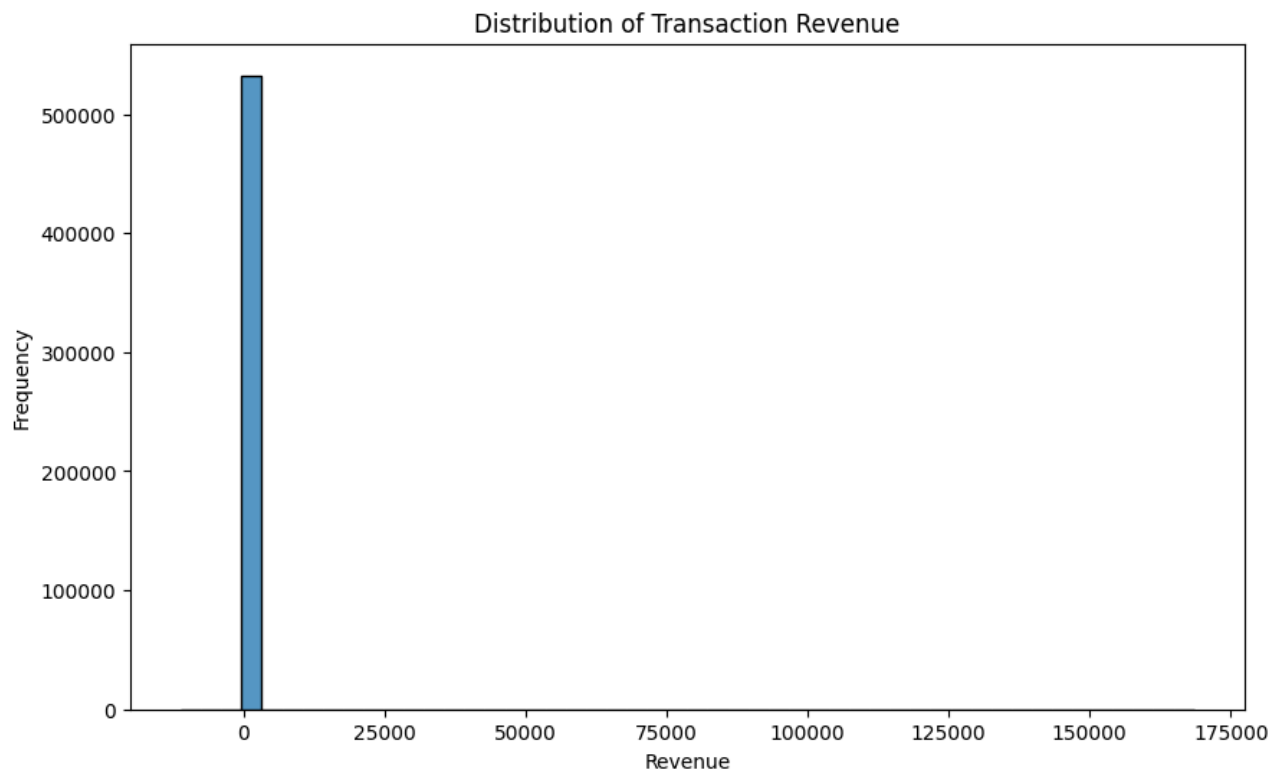
dtype: float64

```
In [ ]: plt.figure(figsize=(10, 6))

sns.histplot(
    df_sales["Revenue"],
    bins=50
)

plt.title("Distribution of Transaction Revenue")
plt.xlabel("Revenue")
plt.ylabel("Frequency")

plt.show()
```



```
In [ ]: # Top 10 Transactions by Revenue

top_transactions = (
    df_sales
    .sort_values("Revenue", ascending=False)
    [[
        "InvoiceNo",
        "StockCode",
        "Description",
        "Quantity",
        "UnitPrice",
        "Revenue",
        "Country"
    ]]
    .head(10)
```

)

top_transactions

Out[]:

	InvoiceNo	StockCode	Description	Quantity	UnitPrice	Revenue	Country
540421	581483	23843	PAPER CRAFT , LITTLE BIRDIE	80995	2.08	168469.60	United Kingdom
61619	541431	23166	MEDIUM CERAMIC TOP STORAGE JAR	74215	1.04	77183.60	United Kingdom
222680	556444	22502	PICNIC BASKET WICKER 60 PIECES	60	649.50	38970.00	United Kingdom
15017	537632	AMAZONFEE	AMAZON FEE	1	13541.33	13541.33	United Kingdom
299982	A563185	B	Adjust bad debt	1	11062.06	11062.06	United Kingdom
173382	551697	POST	POSTAGE	1	8142.75	8142.75	United Kingdom
348325	567423	23243	SET OF TEA COFFEE SUGAR TINS PANTRY	1412	5.06	7144.72	United Kingdom
52711	540815	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2.10	6539.40	United Kingdom
160546	550461	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2.10	6539.40	United Kingdom
421601	573003	23084	RABBIT NIGHT LIGHT	2400	2.08	4992.00	Netherlands

In []: *# Data Quality Summary*

```
quality_summary = pd.DataFrame({  
    "Metric": [  
        "Total Records",  
        "Total Columns",  
        "Missing Customer IDs",  
        "Missing Descriptions",  
        "Duplicate Records",  
        "Cancelled Records",  
    ]  
})
```

```

        "Zero/Negative Quantity",
        "Zero/Negative Unit Price"
    ],
    "Count": [
        len(df),
        len(df.columns),
        df["CustomerID"].isnull().sum(),
        df["Description"].isnull().sum(),
        df.duplicated().sum(),
        cancelled_count,
        (df["Quantity"] <= 0).sum(),
        (df["UnitPrice"] <= 0).sum()
    ]
})

quality_summary

```

Out[]:

	Metric	Count
0	Total Records	541909
1	Total Columns	8
2	Missing Customer IDs	135080
3	Missing Descriptions	1454
4	Duplicate Records	5268
5	Cancelled Records	9288
6	Zero/Negative Quantity	10624
7	Zero/Negative Unit Price	2517

In []:

```

print("Total Revenue:", total_revenue)
print("Total Transactions:", total_transactions)
print("Total Products:", total_products)
print("Total Customers:", total_customers)
print("Repeat Customer %:", repeat_customer_percentage)
print("Cancellation %:", cancellation_percentage)
print("Top 10 Customer Revenue %:", top_10_percentage)

```

```

Total Revenue: 10644560.424000002
Total Transactions: 22064
Total Products: 4059
Total Customers: 4339
Repeat Customer %: 65.56810324959667
Cancellation %: 1.7139409015166756
Top 10 Customer Revenue %: 17.261881248972173

```

In []:

```

print("Highest Revenue Product:")
print(product_analysis.index[0])

print("\nHighest Revenue Country:")
print(country_analysis.index[0])

```

```
print("\nHighest Revenue Month:")
print(monthly_analysis["Revenue"].idxmax())

print("\nLowest Revenue Month:")
print(monthly_analysis["Revenue"].idxmin())
```

Highest Revenue Product:
DOTCOM POSTAGE

Highest Revenue Country:
United Kingdom

Highest Revenue Month:
2011-11

Lowest Revenue Month:
2011-02

```
In [ ]: print("Total Revenue:", total_revenue)
        print("Total Transactions:", total_transactions)
        print("Total Products:", total_products)
        print("Total Customers:", total_customers)
```

Total Revenue: 10644560.424000002
Total Transactions: 22064
Total Products: 4059
Total Customers: 4339

```
In [ ]: print("Repeat Customer %:", repeat_customer_percentage)
        print("Top 10 Customer Revenue %:", top_10_percentage)
```

Repeat Customer %: 65.56810324959667
Top 10 Customer Revenue %: 17.261881248972173

```
In [ ]: print("Cancellation %:", cancellation_percentage)
```

Cancellation %: 1.7139409015166756

```
In [ ]: print(product_analysis.index[0])
        print(country_analysis.index[0])
        print(monthly_analysis["Revenue"].idxmax())
        print(monthly_analysis["Revenue"].idxmin())
```

DOTCOM POSTAGE
United Kingdom
2011-11
2011-02

```
In [ ]: print("Total Records:", len(df))
        print("Total Columns:", len(df.columns))
        print("Sales Records:", len(df_sales))
        print("Total Revenue: £{:.2f}".format(df_sales["Revenue"].sum()))
        print("Total Transactions:", df_sales["InvoiceNo"].nunique())
        print("Unique Products:", df_sales["StockCode"].nunique())
        print("Unique Customers:", df_sales["CustomerID"].nunique())
```

```
print("Average Order Value: {:.2f}".format(
    df_sales.groupby("InvoiceNo")["Revenue"].sum().mean()
))
```

Total Records: 541909
 Total Columns: 8
 Sales Records: 532621
 Total Revenue: £10644560.42
 Total Transactions: 22064
 Unique Products: 4059
 Unique Customers: 4339
 Average Order Value: £482.44

```
In [ ]: quality_summary = pd.DataFrame({
    "Metric": [
        "Total Records",
        "Missing Customer IDs",
        "Missing Descriptions",
        "Duplicate Records",
        "Cancelled Transactions",
        "Non-positive Quantity",
        "Non-positive Unit Price"
    ],
    "Count": [
        len(df),
        df["CustomerID"].isnull().sum(),
        df["Description"].isnull().sum(),
        df.duplicated().sum(),
        cancelled.sum(),
        (df["Quantity"] <= 0).sum(),
        (df["UnitPrice"] <= 0).sum()
    ]
})
```

quality_summary

```
Out[ ]:
```

	Metric	Count
0	Total Records	541909
1	Missing Customer IDs	135080
2	Missing Descriptions	1454
3	Duplicate Records	5268
4	Cancelled Transactions	9288
5	Non-positive Quantity	10624
6	Non-positive Unit Price	2517

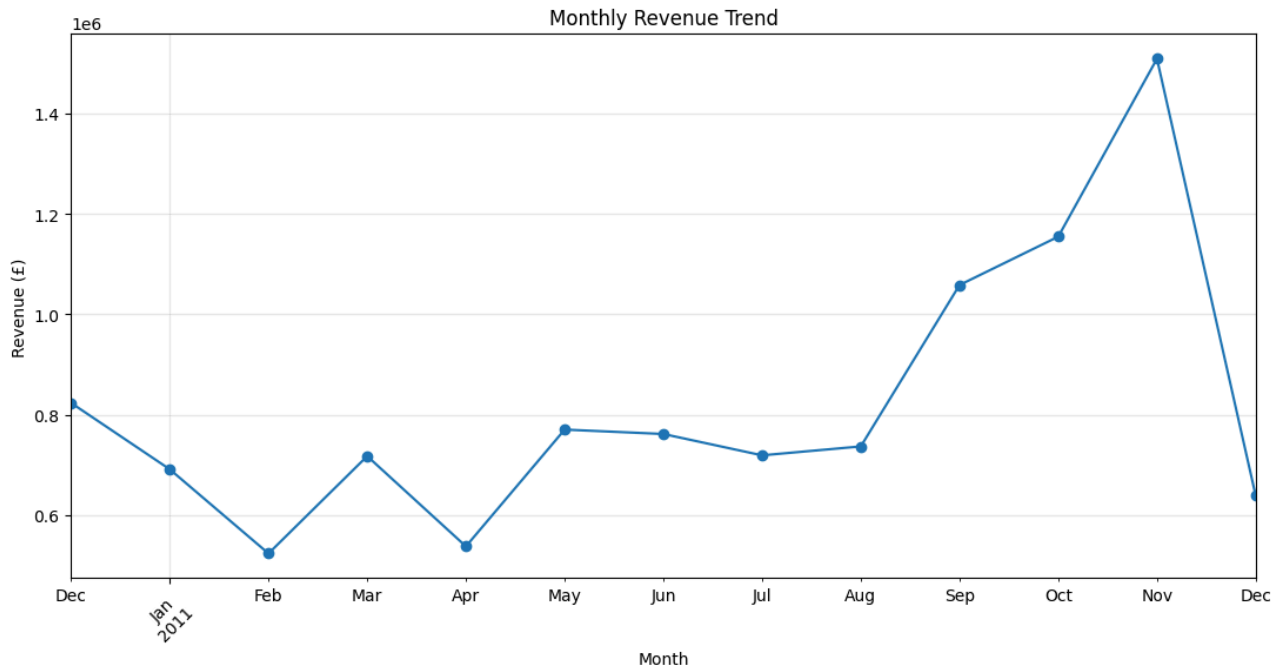
```
In [ ]: monthly_revenue = (
    df_sales
    .groupby(df_sales["InvoiceDate"].dt.to_period("M"))["Revenue"]
    .sum()
```

```
)

plt.figure(figsize=(13, 6))
monthly_revenue.plot(kind="line", marker="o")

plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue (£)")
plt.xticks(rotation=45)
plt.grid(alpha=0.3)

plt.show()
```

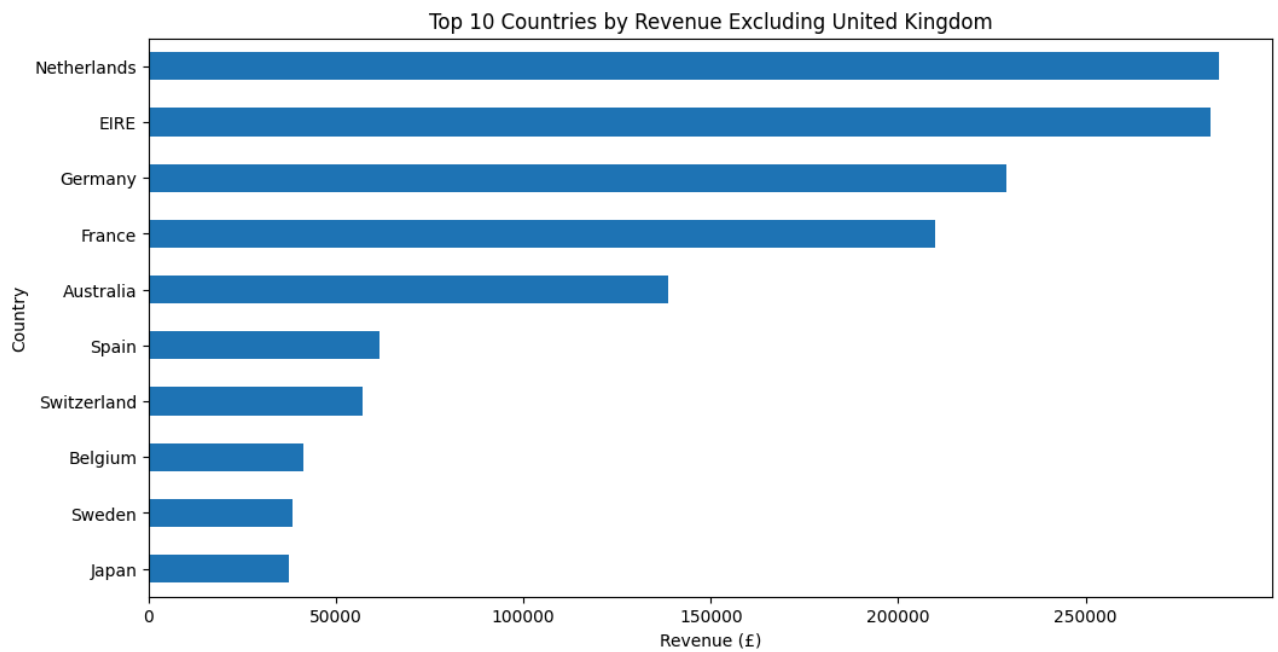


```
In [ ]: country_revenue_non_uk = (
    country_revenue.drop("United Kingdom")
    .head(10)
    .sort_values()
)

plt.figure(figsize=(12, 6))
country_revenue_non_uk.plot(kind="barh")

plt.title("Top 10 Countries by Revenue Excluding United Kingdom")
plt.xlabel("Revenue (£)")
plt.ylabel("Country")

plt.show()
```



```
In [ ]: product_df = df_sales[
    df_sales["Description"].notna() &
    ~df_sales["Description"].str.contains(
        "POSTAGE|MANUAL",
        case=False,
        na=False
    )
].copy()

actual_product_revenue = (
    product_df
    .groupby("Description")["Revenue"]
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

actual_product_revenue
```

Out[]:

	Revenue
Description	
REGENCY CAKESTAND 3 TIER	174484.74
PAPER CRAFT , LITTLE BIRDIE	168469.60
WHITE HANGING HEART T-LIGHT HOLDER	106292.77
PARTY BUNTING	99504.33
JUMBO BAG RED RETROSPOT	94340.05
MEDIUM CERAMIC TOP STORAGE JAR	81700.92
RABBIT NIGHT LIGHT	66964.99
PAPER CHAIN KIT 50'S CHRISTMAS	64952.29
ASSORTED COLOUR BIRD ORNAMENT	59094.93
CHILLI LIGHTS	54117.76

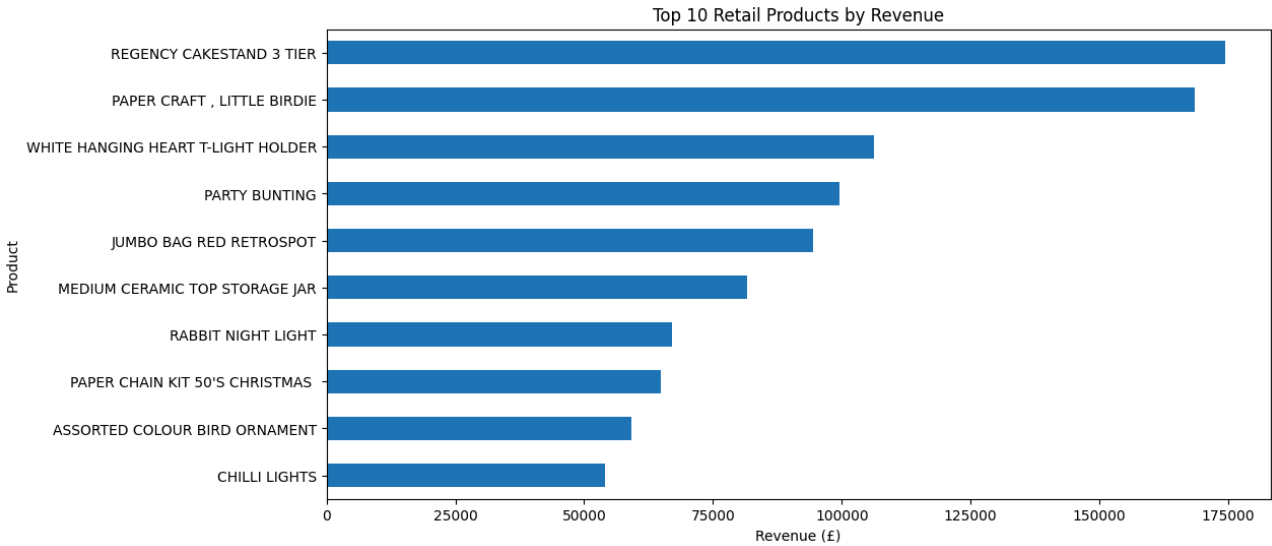
dtype: float64

```
In [ ]: plt.figure(figsize=(12, 6))

actual_product_revenue.sort_values().plot(kind="barh")

plt.title("Top 10 Retail Products by Revenue")
plt.xlabel("Revenue (£)")
plt.ylabel("Product")

plt.show()
```



```
In [ ]: customer_orders = (
    df_sales
    .groupby("CustomerID")["InvoiceNo"]
```

```

        .nunique()
    )

customer_revenue = (
    df_sales
    .groupby("CustomerID")["Revenue"]
    .sum()
)

repeat_customer_percentage = (
    (customer_orders > 1).mean() * 100
)

top_10_customer_percentage = (
    customer_revenue.nlargest(10).sum()
    / customer_revenue.sum()
    * 100
)

print("Repeat Customer Percentage:",
      round(repeat_customer_percentage, 2), "%")

print("Revenue Share of Top 10 Customers:",
      round(top_10_customer_percentage, 2), "%")

```

Repeat Customer Percentage: 65.57 %
 Revenue Share of Top 10 Customers: 17.26 %

```

In [ ]: df_sales[
    [
        "InvoiceNo",
        "StockCode",
        "Description",
        "Quantity",
        "UnitPrice",
        "Revenue",
        "Country"
    ]
].sort_values(
    "Quantity",
    ascending=False
).head(10)

```

Out[]:

	InvoiceNo	StockCode	Description	Quantity	UnitPrice	Revenue	Country
540421	581483	23843	PAPER CRAFT , LITTLE BIRDIE	80995	2.08	168469.60	United Kingdom
61619	541431	23166	MEDIUM CERAMIC TOP STORAGE JAR	74215	1.04	77183.60	United Kingdom
502122	578841	84826	ASSTD DESIGN 3D PAPER STICKERS	12540	0.00	0.00	United Kingdom
74614	542504	37413	NaN	5568	0.00	0.00	United Kingdom
421632	573008	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	4800	0.21	1008.00	United Kingdom
206121	554868	22197	SMALL POPCORN HOLDER	4300	0.72	3096.00	United Kingdom
220843	556231	85123A	?	4000	0.00	0.00	United Kingdom
97432	544612	22053	EMPIRE DESIGN ROSETTE	3906	0.82	3202.92	United Kingdom
270885	560599	18007	ESSENTIAL BALM 3.5g TIN IN ENVELOPE	3186	0.06	191.16	United Kingdom
52711	540815	21108	FAIRY CAKE FLANNEL ASSORTED COLOUR	3114	2.10	6539.40	United Kingdom

```
In [ ]: df_sales.to_csv(  
        "/content/cleaned_online_retail_analysis.csv",  
        index=False  
    )
```

```
In [ ]: print("Highest Revenue Product:")  
        print(actual_product_revenue.index[0])  
  
        print("\nHighest Revenue Country:")  
        print(country_revenue.index[0])  
  
        print("\nHighest Revenue Month:")  
        print(monthly_revenue.idxmax())
```

```
print("\nLowest Revenue Month:")
print(monthly_revenue.idxmin())
```

Highest Revenue Product:
REGENCY CAKESTAND 3 TIER

Highest Revenue Country:
United Kingdom

Highest Revenue Month:
2011-11

Lowest Revenue Month:
2011-02

```
In [ ]: final_findings = pd.DataFrame({
    "Finding": [
        "Total Revenue",
        "Average Order Value",
        "Repeat Customer Percentage",
        "Top 10 Customer Revenue Share",
        "Highest Revenue Product",
        "Highest Revenue Country",
        "Highest Revenue Month",
        "Lowest Revenue Month"
    ],
    "Result": [
        f"£{df_sales['Revenue'].sum():,.2f}",
        f"£{df_sales.groupby('InvoiceNo')['Revenue'].sum().mean():,.2f}",
        f"{repeat_customer_percentage:.2f}%",
        f"{top_10_customer_percentage:.2f}%",
        actual_product_revenue.index[0],
        country_revenue.index[0],
        str(monthly_revenue.idxmax()),
        str(monthly_revenue.idxmin())
    ]
})

final_findings
```

Out[]:

	Finding	Result
0	Total Revenue	£10,644,560.42
1	Average Order Value	£482.44
2	Repeat Customer Percentage	65.57%
3	Top 10 Customer Revenue Share	17.26%
4	Highest Revenue Product	REGENCY CAKESTAND 3 TIER
5	Highest Revenue Country	United Kingdom
6	Highest Revenue Month	2011-11
7	Lowest Revenue Month	2011-02

```
In [ ]: df_sales.to_csv(  
        "/content/cleaned_online_retail_analysis.csv",  
        index=False  
    )  
  
    print("Cleaned analysis dataset saved successfully.")  
Cleaned analysis dataset saved successfully.
```

```
In [ ]: import os  
  
    print(os.path.exists("/content/cleaned_online_retail_analysis.csv"))  
True
```